

Chapter 1: The Modern Federal Identity, Credential, and Access Management (FICAM) Active Directory (AD) Battlespace

Abstract

Key Terminology. Active Directory (AD), Domain Controller (DC), Certificate Authority (CA), Azure AD, Federal Identity, Credential, and Access Management (FICAM), Personal Identity Verification (PIV), Common Access Card (CAC), Public Key Cryptography for Initial Authentication in Kerberos (PKINIT), Active Directory Certificate Services (AD CS), Public Key Infrastructure (PKI), Tier 0, AD Connect/Entra Connect/Entra ID, trust relationship, authentication boundary, trust objects, outbound trust, inbound trust, bidirectional trust, Security Identifier (SID), SID History, Key Distribution Center (KDC), Privilege Attribute Certificate (PAC), forest trust, Golden Ticket, Mimikatz

1.0 Introduction

Every Active Directory (AD) environment you have ever assessed was sold to its owners as “secure” at some point. A consultant ran a hardening project, a checklist got signed, a compliance auditor stamped a report, and everyone went home satisfied. None of that mattered six months later, because AD doesn’t decay gracefully – it decays the way unattended infrastructure always decays: silently, through a thousand small exceptions, until the environment bears no resemblance to the architecture diagram in the SharePoint site nobody reads anymore.

This chapter is not a refresher on what a Domain Controller (DC) does. You know that already. This chapter maps the terrain you’ll actually be fighting on: the trust relationships that nobody audits after the merger, the tiering model that exists on paper but not in Restricted Groups, the Certificate Authority (CA) everyone forgot was a domain-equivalent asset, and the hybrid Azure AD sync server sitting in a server room with the same patch cadence as a print server. Federal Identity, Credential, and Access Management (FICAM)-aligned environments – the federal and military ones, and the contractors who inherited federal control requirements – add Personal Identity Verification (PIV)/Common Access Card (CAC) smart card authentication and Public Key Cryptography for Initial Authentication in Kerberos (PKINIT) to this picture, which sounds like it should raise the bar. Well, it does, but only when implemented correctly. And it almost never is. Not in my 20 years of engaging with Active Directory environments of all flavors and sizes has it ever been correct.

We cover the smart card-specific attack surface in depth in Chapters 2 and 4. Here, we establish why the underlying domain architecture those smart cards plug into is usually already compromised before the card ever becomes relevant.

Four structural realities define the modern AD attack surface. Trust architecture is rarely audited after initial creation. The tiering model, where it exists, is enforced by policy documents instead of technical security controls. Active Directory Certificate Services (AD CS) is treated as a commodity Public Key Infrastructure (PKI) architecture instead of the Tier 0 asset it actually is. And the Azure AD Connect (now Entra Connect) sync server has become the highest-value, least-defended box in most hybrid estates. Walk through all four, and you will understand why “we have MFA and a tiering model” is not considered a security posture – it is a sentence someone says to satisfy and pacify board heads in a board meeting.

1.1 Forest and Domain Trust Architecture as Attack Surface

1.1.1 Trust Mechanics You Need to Stop Ignoring

A trust relationship is not a convenience feature. It is an extension of your authentication boundary into someone else's administrative control, and most organizations treat it like a network firewall rule – set once during a merger or acquisition, never revisited, never de/re-scoped.

Trust objects live in Active Directory as trustedDomain objects under the System container, and every one of them carries an msDS-TrustAttributes value that determines the actual security posture of that trust. The bit flags that matter operationally:

0x00000001	TRUST_ATTRIBUTE_NON_TRANSITIVE	
0x00000002	TRUST_ATTRIBUTE_UPLEVEL_ONLY	
0x00000004	TRUST_ATTRIBUTE_QUARANTINED_DOMAIN	(SID Filtering enabled)
0x00000008	TRUST_ATTRIBUTE_FOREST_TRANSITIVE	
0x00000010	TRUST_ATTRIBUTE_CROSS_ORGANIZATION	(Selective Authentication enabled)
0x00000020	TRUST_ATTRIBUTE_WITHIN_FOREST	
0x00000040	TRUST_ATTRIBUTE_TREAT_AS_EXTERNAL	

Pull these for every trust in an environment before you do anything else on an engagement:

```
Get-ADTrust -Filter * -Properties msDS-TrustAttributes, Direction, TrustType |  
Select-Object Name, Direction, TrustType, msDS-TrustAttributes
```

Direction tells you whether the trust is bidirectional, inbound, or outbound. An outbound trust means the foreign domain trusts you – useless to an attacker sitting in your domain. An inbound trust means you trust the foreign domain, which means a compromised account in that foreign domain can authenticate into yours.

Bidirectional trusts are both at once (like a two-way conversation), and they're disturbingly common because nobody wants to deal with the support tickets that come from scoping a trust down to one direction.

The flag that should make your pulse rise is the absence of 0x00000004. If TRUST_ATTRIBUTE_QUARANTINED_DOMAIN is not set on an inbound, non-forest-internal trust, Security Identifier (SID) filtering is disabled. That single missing bit is the difference between a contained breach in a partner forest and a domain admin ticket forged in your own.

1.1.2 Attack Path: SID History Injection Across Forest Trusts

Security Identifier (SID) History exists for a legitimate reason – migrating users between domains without breaking their access to resources that still reference their old SID. Active Directory tracks this in the sIDHistory attribute on the user object, and during authentication, the Key Distribution Center (KDC) packs every SID in that attribute into the Kerberos Privilege Attribute Certificate (PAC) alongside the user's primary SID. The domain controller validating that PAC checks group membership using the full SID list, sIDHistory included.

Inside a single forest, this is fine – SID filtering doesn't apply within-forest by

design, because Microsoft assumes a forest is a single security boundary (a position you should already be questioning given how many environments have a compromised child domain admin walking straight to Enterprise Admins via the well-documented child-to-parent trust path). Across a forest trust, SID filtering exists specifically to strip out any SID in the PAC that doesn't belong to the trusted domain's own SID namespace. When that filtering is disabled, this is what the path looks like:

1. You compromise Domain Admin (or equivalent) in Forest B, which holds an inbound, non-quarantined trust from Forest A.
 2. You forge a Kerberos ticket (a Golden Ticket, built from the krbtgt hash of Forest B) for a user account, and populate the SIDHistory field of that forged PAC with the SID of Enterprise Admins from Forest A.
 3. Because SID filtering is off on the trust, the domain controller in Forest A honors that injected SID at face value. The PAC validation logic was never designed to question a SID that the trusted domain itself claims belongs to the requesting principal.
 4. You now hold Enterprise Admin rights in Forest A, having never touched a single account or credential native to that forest.
- The tooling for this is not exotic. Mimikatz's `kerberos::golden` command accepts a `/sids:` parameter explicitly for this purpose:

```
mimikatz # kerberos::golden /user:fakeadmin /domain:forestb.local  
/sid:S-1-5-21-XXXXXXXXXX-XXXXXXXXXX-XXXXXXXXXX /krbtgt:<NTLM hash of krbtgt in Forest  
B> /sids:S-1-5-21-YYYYYYYYY-YYYYYYYYY-YYYYYYYYY-519 /ptt
```

Walk through what each flag does, because line-by-line understanding here is the difference between running an attack and understanding why it always works:

- `/user:fakeadmin` - the principal name embedded in the forged ticket. Doesn't need to exist; the KDC trusts the ticket's signature, not a directory lookup, for cross-realm referral processing once it is accepted.
- `/domain:forest.local` and `/sid:` - the domain and domain SID of Forest B, the forest whose krbtgt you have just compromised. This is the forest issuing the ticket.
- `/krbtgt:` - the RC (Rivest Cipher) or AES (Advanced Encryption Standard) key material for the krbtgt account in Forest B. Without this, no forgery is even possible - krbtgt is the trust anchor for every ticket that domain creates and issues.
- `/side:` - the injected SID. -519 is the well-known RID (Relative Identifier) for Enterprise Admins. This SID gets written into the forged PACs SID history field.
- `/ptt` - pass-the-ticket, injecting the forged Ticket Granting Ticket (TGT) directly into the current logon session instead of writing a .kirbi file to disk. Once that ticket is injected, a TGS-REQ (Ticket Granting Service - Request) for any resource within Forest A - including the parent forest's domain controllers - carries Enterprise Admin privilege in its PAC, and Forest A's DC has no native mechanism to reject it, because SID filtering is the only mechanism designed to catch this, and it is off.

Defensive Countermeasure: Trust Hardening

SID filtering should be enabled on every external and forest trust unless you have a documented, signed-off business justification for disabling it – and using “the vendor said it would break something” excuse is not a documented reason, it is an unverified claim you should test in a lab before accepting as absolute truth.

```
netdom trust forestb.local /domain:foresta.local /quarantine:yes  
/usero:Administrator /passwordo:*
```

Selective authentication alone does not stop SID History injection – it restricts which resources a foreign principal can attempt to authenticate to, not what privileges that principal claims to hold once authentication is attempted. Use both controls, not one as a substitute for the other.

Detection matters as much as configuration here. Trust modifications generate Event ID 4706 (trust created) and 4707 (trust removed) on domain controllers, and trust attribute changes land in 5136 (directory service object modified) against the trustedDomain object.

Alert on all three with zero tolerance – a legitimate trust modification is a planned, ticketed change, never a surprise in your Security Information and Event Management (SIEM). For the SID History injection technique specifically, monitor Event ID 4769 (Kerberos service ticket request) for tickets where the requesting account’s PAC contains SID history values that are inconsistent with that account’s actual migration history – Microsoft’s SIDHistory auditing combined with a script comparing POAC contents against the directory’s recorded sIDHistory attribute will catch forged injections that don’t match what the directory has on record, because the forged ticket’s PAC is generated client-side by the attacker’s tooling and is not guaranteed to match what a legitimate migration would have written.

Run an SID History audit against your own forest regardless of trust posture. Any populated sIDHistory attribute that does not map to a documented, completed migration is either forgotten migration cruft (clean it up) or active compromise (escalate immediately).

```
Get-ADUser -Filter {SIDHistory -like "*"} -Properties SIDHistory, SamAccountName |  
Select-Object SamAccountName, SIDHistory
```

1.1.3 Attack Path: Trust Key Extraction for Inter-Realm Ticket Forgery

SID History injection isn’t the only path through a domain trust, and it isn’t even the most direct one at that. Every trust relationship is backed by a shared secret – the inter-realm trust key – stored as a Local Security Account (LSA) on domain controllers on both sides of the trust. Compromise that key directly, and you can forge inter-realm tickets without ever touching either domain’s krbtgt account, because the trust key is the cryptographic material the KDC uses to validate referral tickets crossing the trust boundary.

The trust key lives in the LSA secrets store under a name pattern of \$\$<domain> (however, the exact format depends on trust direction and type), and it is accessible to any process with SeDebugPrivilege and local administrative rights on a domain controller – the same access level that gets you a krbtgt hash via DCSync,

except this path does not require replication rights at all, just local LSA secret access on a DC you've already compromised:

```
mimikatz # lsadump::trust /patch
```

Walk through what `/patch` does specifically: Mimikatz reads the LSA Policy's secret objects, locates the `\$\$`-prefixed trust secrets, and decrypts them using the boot key extracted from the registry's SYSTEM hive – the same boot key mechanism used to decrypt any LSA secret, including service account passwords stored in LSA Secrets for Scheduled Tasks and Internet Information Services (IIS) application pools. The /patch flag specifically defeats an in-memory protection Microsoft added to make casual secret enumeration harder; however, without it, the secrets remain encrypted in the dump output.

With the inter-realm trust key in hand, forge the inter-realm TGT directly – a ticket issued as if by the trusting domain's own KDC, asserting whatever SID set you choose, exactly as in the SID History technique, except now you don't need krbtgt from either side:

```
mimikatz # kerberos::golden /user:Administrator /domain:childdomain.local  
/sid:S-1-5-21-CCCCCCCC-CCCCCCCC-CCCCCCCC  
/sids:S-1-5-21-PPPPPPPP-PPPPPPPP-PPPPPPPP-519 /rc4:<inter-realm trust key>  
/service:krbtgt /target:parentdomain.local /ticket:trust.kirbi
```

The critical difference from the standard golden ticket command: /service:krbtgt and /target:parentdomain.local together instruct Mimikatz to forge the ticket as an inter-realm referral TGT rather than a same-domain TGT, and /rc4: here is the trust key, not a krbtgt hash. Present that forged inter-realm ticket to the target domain's KDC in a TGS-REQ, and the KDC will validate it using the trust key it independently holds – the same key you have just extracted – and issues a service ticket honoring the SID set you specified, parent-forest Enterprise Admin SID included.

Defensive Countermeasure: Trust Key Protection

Credential Guard, enabled via Group Policy at Computer Configuration\Administrative Templates\System\Device Guard\Turn On Virtualization Based Security, moves LSA secret decryption into an isolated virtualization-based security context that lsadump::trust cannot reach even with the full SYSTEM-level access to the host OS (Operating System) – this is the single highest-impact control against this entire technique class, not just trust key theft.

```
Set-ADAuthenticationPolicy -Identity "Tier0-AuthPolicy" -ProtectedUsers $true
```

Rotate trust passwords on a defined schedule rather than treating trust creation as a one-time, set-it-and-forget-it event – netdom trust supports explicit password resets:

```
netdom trust childdomain.local /domain:parentdomain.local /resetonetimepw
```

/usero:Administrator /passwordo:*

Detection-wise, lsadump::trust and equivalent LSA secret extraction tooling require process access to lsass.exe with specific access rights – Sysmon Event ID 10 (ProcessAccess) with GrantedAccess values consistent with full memory read (0x1010 and similar) against lsass.exe, sourced from any process other than known security tooling, is your detection surface here, and it is the same detection surface that catches every other LSASS-targeting credential theft technique in this book. Build it once at the EDR or Sysmon layer; it pays for itself across a dozen different attack paths, not just this one.

1.2 The Tiering Model and Why It Fails in Practice

1.2.1 The Theory vs. The Production Environment

Microsoft’s tier model – Tier 0 (domain controllers, AD CS, identity infrastructure, and anything that can basically compromise them), Tier 1 (servers and applications), Tier 2 (workstations and end-user assets) – is sound in theory and routinely fictional in practice. The model assumes administrative credentials never cross tier boundaries. Production environment violates the assumption constantly, usually through paths nobody designed on purpose.

The classic failures, in order of how often I actually find them on engagements: Shared service accounts. A service account with Tier 0 rights (domain controller service, AD CS service account, backup software with SeBackupPrivilege on a DC) also runs a scheduled task on a Tier 1 application server, because someone reused credentials instead of provisioning a scoped account. Compromise the Tier 1 box, extract the service account’s credentialed material from memory or from a stored task configuration, and you’ve waltzed straight into Tier 0 territory.

Help desk over-permissioning. Help desk staff need to reset passwords and unlock accounts – a legitimate Tier 2-facing function. Instead of delegating that specific right via dscls or the Delegation of Control wizard scoped to an Organizational Unit (OU), someone added the help desk security group to Account Operators or, worse, a custom group nested inside Server Operators, both of which carry rights that reach into Tier 0 territory (Account Operators can modify most non-protected user and group objects, including, historically, the ability to manipulate group membership that intersects with privileged groups if OU delegation is not airtight). GPO link sprawl. A Group Policy Object (GPO) created for a legitimate Tier 1 purpose – say, configuring a logging agent – gets linked at the domain root “for convenience” instead of scoped to the specific OU that needed it. That GPO now applies to domain controllers, too. If the account with GPO Edit rights on that object is a Tier 1-scoped admin, you have just given a Tier 1 credential the ability to push arbitrary SYSTEM-context script execution to Tier 0 via the next GPO refresh cycle.

Remote Desktop Protocol (RDP) and “temporary” cross-tier logons. An admin RDPs from their Tier 1 management jump box straight into a domain controller “just this once” to fix something urgent. That domain controller now caches credential material for a Tier 1-scoped account inside lsass.exe on a Tier 0 asset, and if that admin’s Tier 1 credential is also valid for anything reachable from a lower tier, the entire model has been quietly inverted.

1.2.2 Attack Path: Tier Boundary Violation via Shared Service Accounts

This path doesn't require a zero-day. It requires patience and BloodHound. Start by ingesting the environment with SharpHound, collecting session and Access Control List (ACL) data alongside the standard group membership graph:

```
SharpHound.exe -c All --zipfilename ad_collection
```

The -c All flag pulls every collection method SharpHound supports – group membership, local admin rights, session data, ACLs, GPO links, and trust data – rather than a narrower subset. On a real engagement you'd scope this to avoid tripping detections that key off the volume and pattern of Lightweight Directory Access Protocol (LDAP) queries SharpHound generates, but for understanding the attack path, assume full collection. Load the resulting .zip into BloodHound and run a Cypher query that's more useful than the canned "Shortest Path to Domain Admins" query, because what you actually want is evidence of tier boundary violation specifically – a path where a non-Tier-0 principal has a control relationship reaching a Tier 0 asset:

```
MATCH p=shortestPath(
```

```
(n)-[r:AdminTo|HasSession|MemberOf|GenericAll|GenericWrite|WriteDacl|WriteOwner|AllExtendedRights|ForceChangePassword|AddMember*1..]->(dc:Computer {primarygroupid: 516})
```

```
)  
WHERE NOT n.objectid ENDS WITH '-512'
```

```
RETURN pMATCH p=shortestPath(
```

```
(n)-[r:AdminTo|HasSession|MemberOf|GenericAll|GenericWrite|WriteDacl|WriteOwner|AllExtendedRights|ForceChangePassword|AddMember*1..]->(dc:Computer {primarygroupid: 516})
```

```
)  
WHERE NOT n.objectid ENDS WITH '-512'
```

```
RETURN p
```

This query finds any shortest path from a non-Domain-Admin principal (n, filtered to exclude Domain Admins themselves via the -512 RID exclusion) to a domain controller (primarygroupid: 515 identifies computer accounts in the Domain Controllers group), traversing the relationship edge types most relevant to control: administrative rights, active logon sessions, group nesting, and ACL-based control primitives. When that query surfaces a service account with a session on a Tier 1 SQL server and AdminTo rights on a domain controller – which it will, in most environments large enough to have organically grown service account sprawl – your path is set. Compromise the SQL server (a far softer target than a DC, almost by definition), dump LSASS with a tool of your choice, extract the service account's New Technology Local Area Network Manager (NTLM) hash or Kerberos keys, and use Overpass-the-Hash (Oth) (covered in full protocol detail in Chapter 3) to request a TGT as that

service account directly against the domain controller it has administrative rights on.

```
mimikatz # sekurlsa::pth /user:svc_backup /domain:corp.local /ntlm:<hash>
/run:cmd.exe
```

That spawned cmd.exe carries the service account's NTLM credential material injected into its logon session, valid for NTLM authentication and, via Overpass-the-Hash key derivation, Kerberos authentication too – to anything that account has rights on, including, in this scenario, a domain controller.

Defensive Countermeasure: Enforcing Tier Isolation

Policy documents don't stop lateral movement. Technical controls do. Three mechanisms turn the tier model from aspiration into enforcement:

1. Authentication Policy Siloes

Available since Windows Server 2012 R2 functional level, siloes bind specific accounts to specific authentication policies that restrict which hosts they can authenticate from and to. A Tier 0 admin account assigned to a silo can be technically barred from ever establishing an interactive or network logon session on a Tier 1 or Tier 2 host, which kills the "RDP from a jump box into a DC, just this once" failure mode at the protocol level rather than the policy-document level.

```
New-ADAuthenticationPolicySilo -Name "Tier0-Silo" -Enforce
New-ADAuthenticationPolicy -Name "Tier0-AuthPolicy" `
    -UserTGTLifetimeMins 240 `
    -ComputerTGTLifetimeMins 240
Set-ADAccountAuthenticationPolicySilo -Identity "svc_dc_admin"
-AuthenticationPolicySilo "Tier0-Silo"
Set-ADAccountAuthenticationPolicySilo -Identity "DC01$" -AuthenticationPolicySilo
"Tier0-Silo"
```

Note:

Pay special attention to the -Enforce flag specifically – siloes can be deployed in audit-only mode first (omit -Enforce, monitor the resulting Event ID 4820/4821 logon restriction events), which you should always do before flipping enforcement on in a production forest, because a silo misconfiguration that locks out your actual domain controller admin accounts is a self-inflicted outage nobody thanks you for.

2. Restricted Groups and Explicit Delegation, Never Broad Built-In Group

Membership

Help desk password reset rights get delegated via dsacIs against the specific OU containing Tier 2 user accounts, scoped to the Reset Password extended right and nothing else.

```
New-ADAuthenticationPolicySilo -Name "Tier0-Silo" -Enforce
New-ADAuthenticationPolicy -Name "Tier0-AuthPolicy" `
    -UserTGTLifetimeMins 240 `
    -ComputerTGTLifetimeMins 240
Set-ADAccountAuthenticationPolicySilo -Identity "svc_dc_admin"
    -AuthenticationPolicySilo "Tier0-Silo"
Set-ADAccountAuthenticationPolicySilo -Identity "DC01$" -AuthenticationPolicySilo
    "Tier0-Silo"
```

Audit existing membership in Account Operators, Server Operators, Print Operators, and Backup Operators on every engagement and in every environment you defend – these groups carry privilege that routinely gets treated as “basically nothing” until someone maps out exactly what SeBackupPrivilege actually grants against ntds.dit.

3. GPO Scope Discipline: Enforced Through Delegated GPO Editing Rights That Match Tier B0undaries

A Tier 1-scoped administrator should not hold Edit settings, delete, modify security rights on any GPO linked at the domain root or the Domain Controllers OU. Audit this directly:

```
Get-GPO -All | ForEach-Object {
    $gpoPerms = Get-GPPermission -Guid $_.Id -All
    [PSCustomObject]@{
        GPOName = $_.DisplayName
        Permissions = $gpoPerms | Where-Object {$_.Permission -eq
"GpoEditDeleteModifySecurity"}
    }
} | Where-Object {$_.Permissions}
```

Cross-reference the output against your Tier 0 admin group membership. Any non-Tier-0 principal holding edit rights on a GPO linked anywhere near the Domain Controllers OU is a critical finding, full stop, no exceptions for “but it’s just a logging policy.”

1.2.3 Attack Path: AdminSDHolder ACL Backdoor for Persistent Tier 0 Access

Every protected group in the forest – Domain Admins, Enterprise Admins, Schema Admins, Account Operators, Backup Operators, Server Operators, and a handful of others – has its members’ ACLs forcibly overwritten every 60 minutes by a process called SDProp (Security Descriptor Propagation), running on the Primary Domain Controller Emulator (PDCE). SDProp copies the ACL from a single template object, CN=AdminSDHolder,CN=System,DC=corp,DC=local, onto every member of those protected groups, and sets admincount=1 on each affected object to mark it as protected and

exempt from normal OU-inherited permission changes.

This exists to stop a specific failure mode: an admin grants broad delegated rights on an OU for legitimate reasons, and without SDProp, those rights would silently apply to any privileged account that happened to live in that OU. SDProp's enforcement is the control. It is also the backdoor mechanism, because SDProp doesn't discriminate between a legitimate Access Control Entry (ACE) and one an attacker implanted.

If you gain write access to the AdminSDHolder object itself – even momentarily, even though a privilege escalation path that gets remediated an hour later – you can add an ACE granting a low-profile account full controls, and SDProp will propagate that ACE onto every Domain Admin, every Enterprise Admin, and every other protected-group member in the forest within the next 60-minute cycle, automatically, without any further action on your part:

```
dsaccls "CN=AdminSDHolder,CN=System,DC=corp,DC=local" /G "CORP\svc_helpdesk:GA"
```

/G grants the specified ACE; GA is Generic All. svc_helpdesk here is deliberately chosen to look unremarkable – a service account name that wouldn't draw a second glance in a routine permissions review of Domain Admins membership, because the account never actually gets added to Domain Admins. It doesn't need to. It holds full control over every account that is a member, which is functionally equivalent and considerably harder to spot, because nobody audits "who has GenericAll over the Domain Admins membership" with the same rigor they audit "who is a member of Domain Admins?"

This is a persistence technique more than an initial escalation technique – its value to an attacker is surviving incident response. A responder who finds and removes a compromised account from Domain Admins, rotates that account's password, and calls the incident closed has not touched the AdminSDHolder ACE at all. The backdoor account retains full control over the Domain Admins group and every individual member account, indefinitely, until someone thinks to check the one object most incident response playbooks never look at and always fail to mention.

Defensive Countermeasure: AdminSDHolder Baseline and Monitoring

Baseline the AdminSDHolder ACL in a known-good state and diff against it on a recurring schedule – this object should change rarely, and every change should map to a documented, intentional administrative action:

```
(Get-Acl "AD:\CN=AdminSDHolder,CN=System,DC=corp,DC=local").Access |  
    Select-Object IdentityReference, ActiveDirectoryRights, AccessControlType |  
    Export-Csv -Path "adminsdsdholder_baseline.csv" -NoTypeInfoInformation
```

Re-run that export on a schedule and diff it against the baseline file – any new IdentityReference entry is a finding requiring immediate investigation, not a ticket for next sprint.

Directly alert on modification of the AdminSDHolder object itself. Object modification against this specific Distinguished Name (DN) generates Event ID 5136

(directory service object modified) on the domain controller holding the PDC Emulator role at minimum, and ideally across all DCs with the appropriate audit policy enabled (Audit Directory Service Changes under Computer Configuration\Windows Settings\Security Settings\Advanced Audit Policy Configuration\DS Access). Treat any 5136 events against CN=AdminSDHolder as a Tier 0 incident requiring immediate triage – there is no benign reason for this object to change outside a planned administrative action you already know about.

Finally, audit admincount=1 itself as a standing control, independent of AdminSDHolder ACL review. Accounts marked admincount=1 that are no longer actual members of any protected group are common – people get removed from Domain Admins and the admincount flag doesn't automatically clear – but each one represents an account that was at one point in time protected, may still carry residual permissions inherited from its protection period, and deserves explicit review rather than being dismissed as cleanup debt.

```
Get-ADUser -LDAPFilter "(adminCount=1)" -Properties adminCount, MemberOf |  
    Select-Object SamAccountName, MemberOf
```

Cross-reference every account this query returns against current protected group membership. Anything flagged admincount=1 with no corresponding current membership in a protected group is either forgotten cleanup or an active backdoor – and you don't get to assume which without first checking.

1.3 Active Directory Certificate Services (AD CS) as the Silent Kingmaker

1.3.1 Why Nobody Treats the CA Like Tier 0

Active Directory Certificate Services (AD CS) gets deployed, configured once during a project with a tight deadline, and then forgotten about after everything is said and done. It sits in inventory as “the PKI server” – a categorization that, in most asset management systems, places it somewhere between a file server and a print server in terms of perceived criticality. That categorization is dead wrong, and it is wrong in a way that has handed more red teams a domain than almost any other single misconfiguration class in the last decade.

Here's the structural reality: a certificate authority's private key is a forging credential for identity itself. If you control the CA, you can mint a certificate asserting any identity of your choosing – including Domain Admin, including a domain controller's own machine identity – and that certificate will be honored by every system in the forest that trusts the CA, which is practically every domain-joined system by default, because CA trust gets distributed automatically via Group Policy into every computer's NTAUTH store and trusted root store. The CA is not “PKI infrastructure adjacent to Tier 0.” The CA is Tier 0, definitionally, and any environment where the CA's administrative group does not map one-to-one with Tier 0 admin membership has already failed the tiering model discussed in section 1.2, whether anyone has noticed yet or not.

Full exploitation mechanics for each AD CS escalation path – the ESC1 through ESC16 family of misconfigurations – get their own complete treatment in Chapter 5, including certificate template attribute analysis, web enrollment agent abuse, and NTLM relay into HyperText Transfer Protocol (HTTP) enrollment endpoints. What belongs here is the architectural point I'm trying to drive: AD CS misconfiguration

isn't a niche finding. It is the absolute default state of most CA's deployed before 2021, because the certificate template permission model is genuinely difficult to reason about, and Microsoft's default templates shipped with permission combinations that, in retrospect, should never have been the default.

To begin, it is important to first establish a general understanding of how Active Directory identity compromise integrates with the MITRE ATT&CK Framework. Active Directory (AD) identity compromise is the process where an adversary obtains, abuses, or manipulates credentials and privileges within an AD environment. The MITRE ATT&CK Framework maps these adversarial behaviors across the attack lifecycle, primarily focusing on how attackers gain initial access, elevate privileges, and maintain persistent control within a domain.

Key MITRE ATT&CK Tactics & Techniques

Adversaries typically target AD identities across three core phases of the attack lifecycle:

1. Credential Access (TA0006)
 - OS Credential Dumping (T1003): Attackers often target the LSASS process memory or directly dump the Active Directory database (`ntds.dit`) to steal password hashes and Kerberos tickets.
 - Steal or Forge Kerberos Tickets (T1558): Attackers abuse Kerberos mechanics to move laterally, notably via Golden Tickets (forging Ticket Granting Tickets (TGTs)) or Silver Tickets (forging Ticket Granting Service (TGS) tickets).
 - Unsecured Credentials (T1552): Finding credentials stored in plaintext or configuration files, such as Group Policy Preferences (GPP) `cpasswords`.
2. Privilege Escalation (TA0004)
 - Valid Accounts (T1078.002): Using legitimately acquired or cracked domain accounts (user, admin, or service accounts) to access resources.
 - Kerberoasting (T1558.003): Requesting Kerberos Ticket Granting Service (TGS) tickets for Service Principal Names (SPNs) to crack service account passwords offline.
 - DCSync (T1003.006): Mimicking a Domain Controller to synchronize password data, stealthily extracting password hashes for all domain users.
3. Persistence & Defense Evasion (TA0003/TA0005)
 - Domain or Tenant Policy Modification (T1484): Modifying Group Policy Objects (GPOs) to execute malicious payloads or alter domain configurations.
 - Account Manipulation (T1098): Modifying account attributes (like adding a user to a privileged group) or injecting malicious authentication processes (e.g., modifying Active Directory Federation Services (AD FS)) to establish backdoor access.

1.3.2 Attack Path: Enumeration to Domain Compromise

You don't need elevated rights to find out whether a CA is exploitable – template enumeration is where it's at and is available to any authenticated user by design, because certificate enrollment has to be discoverable. This is the first thing you must run on any engagement touching an AD environment with AD CS deployed, full stop, before you have done anything else.

Certify, the most widely used tool for this specific type of reconnaissance, enumerates every certificate template in the forest and flags the ones with dangerous permission or configuration combinations:

Certify.exe find /vulnerable

Walk through what this actually does under the hood, because understanding the query matters more than running the binary. Certify pulls every pKICertificateTemplate object from the Configuration partition (CN=Certificate Templates,CN=Public Key Services,CN=Services,CN=Configuration,DC=corp,DC=local), reads each template's msPKI-Certificate-Name-Flag, msPKI-Enrollment-Flag, pKIExtendedKeyUsage, and the template's discretionary ACL (DACL), then cross-references against the issuing CA's own enrollment permissions. The /vulnerable flag applies Certify's built-in logic for flagging the classic dangerous combinations – most notably a template where ENROLLEE_SUPPLIES_SUBJECT is set in msPKI-Certificate-Name-Flag (meaning the requestor, not the CA, controls the Subject Alternative Name (SAN) on the issued certificate) combined with Client Authentication EKU (Extended Key Usage) and enrollment rights granted to a low-privileged principal like Domain Users or Authenticated Users.

That combination is the entirety of the ESC1 attack class, and its consequence is direct: any domain user can request a certificate from that template, specify an arbitrary SAN – including the User Principal Name (UPN) of a Domain Admin – and receive a valid, CA-signed certificate asserting that identity. PKINIT smart card logon (detailed protocol-level in Chapter 4) accepts that certificate for authentication, because PKINIT trusts the CA's signature, not an independent verification that the requestor actually is the identity named in the SAN.

```
Certify.exe request /ca:dc01.corp.local\corp-CA /template:VulnerableUserTemplate  
/altname:administrator@corp.local
```

The output is a base64-encoded PFX (Personal Information Exchange) certificate encoded data. Convert it and use it directly with the tool, Rubeus, to request a TGT via PKINIT for the impersonated identity:

```
Rubeus.exe asktgt /user:administrator /certificate:<base64 PFX> /password:<PFX  
export password> /ptt
```

That single command chain – enumerate, request, convert, authenticate – takes under five minutes against a vulnerable template and ends with a Domain Admin TGT injected into the current session. No password spraying, no credential theft, no lateral movement. The misconfiguration is the entire attack.

Defensive Countermeasure: Treating the CA Like the Crown Jewel It Is
Audit every certificate template in the forest for the ESC1 pattern specifically – ENROLLEE_SUPPLIES_SUBJECT combined with broad enrollment rights and a client

authentication EKU – as a standing, recurring control, not a one-time project:

```
Get-ADObject -SearchBase "CN=Certificate Templates,CN=Public Key  
Services,CN=Services,CN=Configuration,DC=corp,DC=local"  
-Filter * -Properties msPKI-Certificate-Name-Flag, msPKI-Enrollment-Flag,  
pKIExtendedKeyUsage |  
Where-Object { $_."msPKI-Certificate-Name-Flag" -band 0x1 } |  
Select-Object Name, msPKI-Certificate-Name-Flag, pKIExtendedKeyUsage
```

The 0x1 bitmask checked against msPKI-Certificate-Name-Flag isolates the CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT bit. Any template returned by this query needs immediate review of its enrollment permissions – and in almost every case I’ve witnessed, the fix is removing ENROLLEE_SUPPLIES_SUBJECT entirely for templates that do not have a legitimate operations reason to need it (and most don’t; it exists for narrow scenarios like certain web server enrollment workflows where the CA administrator has a separate process for validating the requested subject). CA-level permissions need the same scrutiny as domain controller administrative group memberships, because they carry equivalent power:

```
certutil -config "dc01.corp.local\corp-CA" -getreg "CA\Security"
```

ManageCA and ManageCertificates rights on the CA object grant the ability to approve pending certificate requests, issue certificates outside the template-defined constraints in certain configurations and modify the CA’s own security settings. These rights should map exactly to your Tier 0 administrative group – no exceptions, no “the PKI team has their own group that’s mostly the same people.” Finally, run the PSPKIAudit tool as a standing compliance checker rather than a one-off assessment tool – it automates detection of the full ESC1-ESC8 misconfiguration family and produces a report suitable for tracking remedial efforts over time, which matters because certificate template sprawl is a continuous problem, not a project with an end date.

```
Import-Module PSPKIAudit  
Invoke-PKIAudit -Targets ADCS -Verbose
```

Log every certificate issuance. CA auditing isn’t enabled by default for all event categories – turn on issuance and revocation auditing explicitly using the Windows operating system built-in command-line tool, certutil.exe:

```
certutil -setreg CA\AuditFilter 127
```

The value 127 enables all seven CA auditable event categories (start/stop,

backup/restore, certificate issuance and management, revocation, security settings changes, key recovery, and CA configuration changes), and the resulting events land as Event ID 4886-4899 range on the CA server itself. Alert on any certificates issued with a SAN matching a privileged account's UPN, where the requesting principal isn't the CA service account or an explicitly authorized enrollment agent – that single correlation rule catches ESC1 exploitation as it happens, not after the fact in a post-incident tabletop review.

1.3.3 Attack Path: Certificate Authority (CA) Private Key Extraction via Backup Abuse

Template misconfiguration is the loud failure mode. The quiet one – the one that should really keep you up at night as a defender – is private key exposure on the CA host itself. The amount of dispersed AD CS deployments outside large federal and financial environments run on a software Cryptographic Service Provider (CSP) rather than a Hardware Security Module (HSM), despite Microsoft's own deployment guidance recommending HSM-backed key storage for any production CA. Software-protected keys are exportable under the right conditions, and "the right conditions" usually just means local administrative rights on the CA server – access most environments hand to a general Windows server admin team that has no idea they're sitting on the forest's root of trust.

Any principal holding the `ManageCA` right, or local admin on the CA host, can attempt a key backup through the standard, fully-supported CA administration interface:

```
certutil -backupkey -p "TempBackupPwd123!" C:\catemp
```

Walk through this precisely. `-backupkey` invokes the CA's built-in key export routine – the same mechanism legitimate CA administrators use to create disaster recovery backups, which means this isn't an exploit in the technical sense at all. It is a documented administrative function being used by someone who was never supposed to hold the rights to call it. `-p`` supplies the password protecting the resulting PFX file, and ``C:\catemp`` is the output path. If the CA's key was provisioned as exportable in the CSP – common, because exportability is frequently left on by default during CA installation so administrators can perform exactly this kind of backup without thinking through who else might one day hold the rights to do it – the command succeeds and drops a PFX containing the full private key.

Where the CSP marks the key non-exportable, Mimikatz's ``crypto::capi`` and ``crypto::cng`` modules patch the CryptoAPI and CNG providers in memory to strip the non-exportable flag enforcement, then reattempt the export:

```
mimikatz # crypto::capi
```

```
mimikatz # crypto::cng
```

```
mimikatz # crypto::certificates /export /systemstore:CA
```

``crypto::capi`` patches the legacy CryptoAPI provider in the context of the current process to ignore the ``CRYPT_EXPORTABLE`` flag check. ``crypto::cng`` does the equivalent for the newer Cryptography API: Next Generation provides most modern Windows Server CAs use. ``crypto::certificates /export /systemstore:CA`` then walks the local machine's CA certificate store and exports every certificate – including where the patch succeeded, the private key material the provider would otherwise have refused to release.

A successfully exported CA private key is the single worst outcome covered in this chapter – worse than any individual Domain Admin compromise, I believe, because it allows an attacker to mint certificates entirely offline. In Active Directory Certificate Services (AD CS) if an attacker steals the Root or Issuing CA's private

key (DPERSIST1 tradecraft), they can use tools like ForgeCert to generate valid certificates completely offline on a non-domain-joined machine. This offline forging completely bypasses the standard network protocols and telemetry defenders rely on. Because the entire forging process occurs outside the live environment, no network enrollment requests traveling over `MS-WCCE` are initiated, no Active Directory Certificate Services (AD CS) audit log entries are generated, and no template constraints are enforced. Post-theft, the attacker requires zero interaction with the CA server, rendering traditional CA monitoring completely blind to the creation of the forge. Consequently, defenders cannot detect the compromise at the CA level; their first and only opportunity for detection shifts entirely to the Domain Controllers, specifically by hunting for anomalous Kerberos PKINIT authentication traffic (such as Event ID 4768) when the forged certificate is fully weaponized to access the domain.

Attack Vector: Key Extraction and DPAPI Bypass

To execute an offline forging attack, an attacker must first extract the CA's private key from the registry or file system. On a standard, software-protected Certificate Authority, this key is encrypted and protected by the Data Protection API (DPAPI) or a software Key Storage Provider (KSP) such as the Microsoft Software Key Storage Provider.

If an attacker achieves local administrative privileges on the CA server, they can elevate to `NT AUTHORITY\SYSTEM` and bypass these software protections. Using tools like Mimikatz or specialized PowerShell scripts, the attacker interacts with the CryptoAPI or CNG (Cryptography Next Generation) architectures. They force the export of non-exportable private keys by patching the memory of the LSASS process or by utilizing specialized APIs to read the raw, obfuscated keys directly from the `HKEY\_LOCAL\_MACHINE\SOFTWARE\Microsoft\SystemCertificates` registry hives or the machine's crypto folder keys. Once extracted into a `.pfx` or `.key` file, the attacker transfers it to their offline rig, completely breaking the security boundary which equals a critical and complete breakdown of implied and enforced security principles that demand immediate review.

Defensive Countermeasure: Hardware Security Modules (HSMs) and CA Key Protection

The single most effective mitigation against CA private key theft is the mandatory enforcement of a Hardware Security Module (HSM). An HSM is a physical, tamper-resistant cryptographic hardware appliance (or a dedicated cloud equivalent) that shifts the storage and utilization of the private key completely out of the software domain.

HSM-backed key storage is the control that permanently closes this attack path rather than just making it marginally harder. A FIPS 140-2 Level 2 or 3 validated HSM holds the CA's private key in tamper-resistant hardware and performs signing operations internally – the key material never exists in a form any software-level access, including full local administrator rights on the CA host, can extract. For a FICAM-aligned environment specifically, HAM-backed PKI isn't a recommendation; treat it as a hard requirement, because the federal PKI trust model this book covers in Chapter 2 assumes a CA's signing key has exactly this property.

When configured correctly with a hardware KSP:

- **True Non-Exportability:** The CA's root private key is generated inside the physical HSM chips and physically cannot be extracted, exported, or even read by the Windows operating system – even if an attacker gains full `SYSTEM` or Domain Admin access of the CA host server.
- **In-Hardware Cryptography:** When the CA needs to sign a certificate or a

Certificate Revocation List (CRL), the data is first sent to the HSM. The cryptographic signing operation occurs inside the secure hardware boundary, and only the finished cryptographic signature is returned to the OS.

- **Render Offline Forgery Impossible:** Because the private key cannot leave the physical hardware appliance, tools like ForgeCert are rendered entirely useless. An attacker cannot stand up an offline machine and mint certificates because they lack access to the physical microchips containing the root key.

Where HSM deployment isn't immediately feasible, restrict `ManageCA` rights with the same discipline applied to Domain Admins group membership, and audit it using the same cadence:

```
certutil -config "dc01.corp.local\corp-CA" -getreg "CA\Security"
```

Disable key exporting capabilities at the CSP/KSP configuration level during CA provisioning – this doesn't stop the Mimikatz in-memory patch technique, but it does close the `certutil -backupkey` path for any principal who hasn't escalated to LSASS-level access, which, in turn, raises the bar from "has ManagedCA rights" to "has achieved full SYSTEM-equivalent compromise of the CA host."

Detection here lives at the process and command-line level. Alert on any execution of `certutil.exe` with the `-backupkey` argument outside a documented. Scheduled CA backup maintenance window – this is rare enough in legitimate operations that the false-positive rate on this single detection is close to zero:

```
ProcessName=certutil.exe AND CommandLine CONTAINS "-backupkey" AND NOT ScheduledMaintenanceWindow
```

Pair that with Sysmon Event ID 10 monitoring for process access against `lsass.exe` and against the CA service process (`certsvc.exe`) – the in-memory CryptoAPI/CNG patching technique requires injecting into or hooking a process holding the key handle, and that access pattern is visible at the EDR layer the same way every other LSASS-adjacent credential theft technique in this chapter is visible, provided you've built the detection instead of assuming your EDR vendor ships it by default (because most do not).

1.4 Hybrid Identity: Azure AD/Entra ID Sync as Expanded Attack Surface

1.4.1 The Sync Server Is a Domain Controller You Forgot to Lock

Microsoft's Entra Connect (the current name for what most of the industry still calls Azure AD Connect, and what most environments still have running under that older branding regardless of what Microsoft calls it this quarter) holds two categories of credential material that, combined, make the sync server functionally equivalent to that of a domain controller from an attacker's perspective, while almost never receiving domain-controller-grade physical, network, or even administrative hardening.

First: the on-premises Active Directory Domain Services (AD DS) Connector account, historically provisioned with a name pattern like MSOL_<random>, holds replication rights sufficient to perform a DCSync attack against your domain – because that is exactly what password hash synchronization is, architecturally speaking. The sync engine needs Replicating Directory Changes and Replicating Directory Changes All extended rights to pull password hash data for synchronization to Entra ID, and those are the same two extended rights that define DCSync capability for any other principal that holds them.

Second: the sync server's local SQL Server instance (or the local LocalDB instance in default configurations) stores the encrypted credentials for that connector account, along with the encryption keys needed to decrypt them, on the local file system – because the sync engine needs to use those credentials autonomously, on a

schedule, without a human present to type a password.

Compromise local administrator rights on the sync server, and both of those facts combine into full domain compromise without ever touching a domain controller directly.

1.4.2 Attack Path: AAD Connect Credential Extraction

The extraction logic has been public since Adam Chester and others reverse-engineered the encryption scheme used by the sync engine's credential storage. The encrypted connector account password lives in the `mms_management_agent` table inside the ADSync database, and it is encrypted using the Windows operating system built-in cryptographic service Data Encryption Application Programming Interface) or DPAPI tied to the sync service's own machine context – which means any process running with local administrator rights on the sync server can decrypt it, because DPAPI in this configuration uses machine-bound keys, not user-bound keys, specifically so the unattended sync service can decrypt its own stored credentials without a logged-in user.

The publicly available tooling for this – ADSyncDecrypt, and the credential-extraction modules built into broader AD assessment frameworks – automates the database query and DPAPI unwrap in one step. The mechanical sequence performs as such:

1. Query the local ADSync SQL instance for the encrypted credential blob stored against the AD DS Connector configuration row.
2. Call `CryptUnprotectData` (the DPAPI decryption API) against that blob, using the machine context – which succeeds because the calling process, running as local admin on the sync server, has access to the machine's DPAPI master key material.
3. Parse the decrypted output, which yields the connector account's plaintext password in a structured format the sync engine itself uses internally.

Once you hold that connector account's plaintext credential, DCSync is a direct, single command away – no further exploitation required, because the account already holds the replication rights by design:

```
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
```

That single line extracts the krbtgt account's key material directly from the domain, using the stolen sync account's legitimately-granted replication rights – which means this isn't an exploit of a vulnerability in the traditional sense. It is the intended function of the sync architecture, reached via a credential theft step that domain-controller-grade hardening would have prevented entirely.

1.4.3 Attack Path: Seamless Single Sign-On (SSO) Computer Account Abuse

Organizations using Azure AD Seamless Single Sign-On (SSO) have a second, independent exposure to worry about: a computer account named `AZUREADSSOACC$` exists in the on-premises domain specifically to support Kerberos-based silent authentication to Entra ID. That computer account's password – never rotated by default in a meaningful number of deployed environments, despite Microsoft's documented 30-day rotational guidance – is shared with Entra ID as a Kerberos decryption key for tickets used in the Seamless SSO flow.

Dirk-jan Mollema's published research on this mechanism demonstrated that an

attacker who extracts the AZUREADSSOACC\$ computer account's NTLM hash (via DCSync, or via any other credential extraction technique that reaches a domain-joined computer account's hash) can forge Kerberos tickets that Entra ID's Seamless SSO validation logic will accept as legitimate silent-authentication assertions for any user in the synchronized tenant – effectively a Golden Ticket equivalent that operates against the cloud identity plane instead of the on-premises domain.

```
mimikatz # lsadump::dcsync /domain:corp.local /user:AZUREADSSOACC$
```

Extracting that single computer account's hash, combined with the forgery tooling build for this specific technique, produces a path from on-premises domain compromise directly into cloud tenant impersonation – collapsing the boundary that organizations frequently assume exists between “on-prem AD got popped” and “our cloud tenant is fine because it is a separate system.”

1.4.4 Attack Path: Malicious Pass-Through Authentication (PTA) Agent Credential Capture

Commands that chose Pass-Through Authentication over password hash sync – often specifically because PTA avoids storing password hash material in the cloud, a decision some FICAM-aligned environments make for compliance reasons – have different exposure that's arguably worse, because it does not require extracting anything from a database. It requires sitting in the path of live, plaintext credentials as real user authentication in real-time.

PTA works by relaying authentication requests from Entra ID, through an outbound connection to an internal Service Bus listener, to an on-premises PTA Agent, which validates the submitted credential directly against on-prem AD and returns a pass/fail result. The agent receives the user's password in a form it can validate against AD – meaning at some point in that validation flow, the agent process holds plaintext-equivalent credential material for every single sign-in attempt routed through it, for as long as that organization has chosen PTA as its sign-in method.

Two distinct attacker positions exploit this:

Position one – local compromise of an existing PTA Agent host. An attacker with local administrative rights on a server running the Azure AD Connect Authentication Agent can hook the agent process to log every credential it validates, capturing plaintext passwords for every user who authenticates through that agent going forward – not a one-time extraction, but an ongoing collection capability that persists until someone finds it.

Position two – registration of a rogue, attacker-controlled agent. Multiple PTA Agents can be registered against a single tenant for high availability, and Entra ID load-balances live authentication requests across all registered agents. An attacker who has obtained Hybrid Identity Administrator or Global Administrator rights in the cloud tenant – itself a serious compromise, but one that doesn't necessarily include any foothold in the on-premises domain – can install and register an additional PTA Agent on infrastructure they control entirely, using the standard, Microsoft-supported agent installer and the compromised admin credential to complete registration:

```
Install-Module -Name AADConnectAuthAgent
```

```
Register-AADConnectAuthAgent -ConnectorName "corp-pta-agent-redundant" -Credential (Get-Credential)
```

Once registered, that rogue agent receives a share of live authentication traffic for the tenant – including, depending on how the load balancing distributes requests, credentials belonging to on-premises accounts the attacker has never otherwise touched, harvested in plaintext as those users go about routine sign-ins with no indication that anything is wrong, because from the user’s perspective, authentication succeeds exactly as expected.

Defensive Countermeasure: PTA Agent Hardening

Every PTA Agent host gets Tier 0 placement, identical to the full Entra Connect sync server discussed in section 1.4.1 – same authentication silo membership, same restricted local administrator group, same physical and network segmentation. There is no meaningful security difference between a PTA Agent server and a domain controller from a risk perspective; the asset inventory and tiering documentation should reflect that without exception.

Monitor agent registration events at the cloud control plane-level. Entra IDs sign-in and audit logs record every new Authentication Agent registration as a distinct, queryable event – alert on any registration that does not correspond to a documented, change-ticketed deployment:

```
Get-MgAuditLogDirectoryAudit -Filter "activityDisplayName eq 'Add Authentication Agent'" |
```

```
    Select-Object ActivityDateTime, InitiatedBy, TargetResources
```

Restrict Hybrid Identity Administrator and Global Administrator role assignment with the same rigor as Domain Admins group membership – the rogue-agent registration path requires one of these roles, and a tenant where either role is held by more than a small, named MFA-enforced set of accounts has already failed this control before an attacker ever thinks to attempt it.

At the host level, deploy EDR coverage on PTA Agent servers explicitly configured to flag process injection or API hooking targeting the Authentication Agent service binary (`AzureADConnectAuthenticationAgentService.exe`) – the local-compromise variant of this attack requires exactly that kind of in-process hooking to intercept credentials as they pass through validation, and it’s detectable the same way LSASS injection is detectable, provided the detection rule names this specific process rather than relying on generic LSASS-only coverage that won’t catch hooking against a different target binary entirely.

Recognize, finally, that there is no credential-validation method here that eliminates risk outright – password hash sync carries the DCSync-adjacent exposure covered in section 1.4.2, and PTA carries this one. The decision between them is a decision about which risk you are better equipped to monitor and defend, not a decision between a secure option and an insecure one. Document that tradeoff explicitly in your architecture decisions rather than treating either method as a checkbox compliance answer.

Defensive Countermeasure: Hardening the Hybrid Identity Plane

The sync servers get domain-controller-grade physical and administrative controls, not server-grade ones, full stop. That means:

Tier 0 placement, explicitly. The Entra Connect server belongs in your authentication silo configuration alongside domain controllers, with local administrator rights restricted to the same group that holds DC admin rights. If your tiering documentation doesn’t already say this, fix the documentation today and the actual group membership before today ends. Seriously.

Connector account credential rotation and least privilege review. Audit the AD DS Connector account’s actual rights – Microsoft’s documented minimum for password hash

sync doesn't require the full Replicating Directory Changes All extended right in every deployment configuration, and where Pass-Through Authentication is used instead of password hash sync, the replication right requirement changes. Confirm what your actual sync method requires and strip anything broader:

```
Get-ADUser -Filter {SamAccountName -like "MSOL_*"} -Properties * |  
    Select-Object SamAccountName, DistinguishedName
```

Another important step is to cross-reference that account for effective rights against the domain's root object ACL and specifically hunt for these rights (again, no one checks): DS-Replication-Get-Changes and DS-Replication-Get-Changes-All extended rights Globally Unique Identifiers (GUIDs) (1131f6aa-9c07-11d1-f79f-00c04fc2dcd2 and 1131f6ad-9c07-11d1-f79f-00c04fc2dcd2 respectively):

```
$rootDSE = Get-ADRootDSE  
(Get-Acl "AD:\ $($rootDSE.defaultNamingContext)").Access |  
    Where-Object { $_.ObjectType -in  
@("1131f6aa-9c07-11d1-f79f-00c04fc2dcd2", "1131f6ad-9c07-11d1-f79f-00c04fc2dcd2") } |  
    Select-Object IdentityReference, ObjectType, ActiveDirectoryRights
```

Any identity in that output beyond your domain controllers, your sync account (scoped appropriately), and explicitly documented backup/replication tooling is a finding requiring immediate explanation. AZUREADSSOACC\$ password rotation, enforced on Microsoft's documented schedule, not left as a manual task someone forgets. Microsoft ships a PowerShell module specifically for this:

```
Import-Module -Name "C:\Program Files\Microsoft Azure Active Directory  
Connect\AzureADSSO.psd1"  
$creds = Get-Credential  
New-AzureADSSOAuthenticationContext  
Update-AzureADSSOForest -OnPremCredentials $creds
```

Schedule this as a recurring task with monitoring on its actual execution – a failed silent rotation that nobody notices defeats the entire control. Detection at the SQL layer. The ADSync database on the sync server should be monitored for any access patterns outside the sync service's own scheduled process – direct sqlcmd or SQL Server Management Studio (SSMS) connections to the local ADSync instance from any account other than the service account itself is a strong indicator of the extraction technique described above, and that monitoring is cheap to implement via SQL Server's native audit specification against the relevant database.

On-premises detection for the actual DCSync call. Regardless of which path led to the stolen credential – sync server compromise, any other route to replication rights – the DCSync call itself generates Event ID 4662 on domain controllers with the object access correlating to the DS-Replication-Get-Changes-All GUID referenced above. Build the correlation rule once, and it catches every variant of this attack path, not just the hybrid-identity-specific one:

```
EventID=4662 AND ObjectType={1131f6ad-9c07-11d1-f79f-00c04fc2dcd2} AND  
SubjectUserName NOT IN (KnownDCAccounts, KnownSyncAccount)
```

That single rule is one of the highest-value detections you will ever deploy in an AD environment. Implement it before you implement almost anything else in this book.

1.5 Chapter Summary

Four structural weaknesses define the terrain: trust relationships that extend your security boundary further than anyone ever audited, a tiering model enforced by policy documents instead of authentication siloes and scoped delegation, a certificate authority treated as commodity infrastructure when it is actually a forging press for domain identity, and a hybrid sync server holding domain-controller-equivalent credential material with server-grade physical, network, and administrative controls.

None of these are “hacker-exotic.” None require a zero-day or require that you have “1337 hacking skills.” Every attack path walked through in this chapter uses publicly available tooling against misconfigurations that exist in the overwhelming majority of production AD environments (FICAM-aligned or not) and have been present in every single AD environment I’ve tested against. The smart card and PKINIT layer that FICAM environments add on top of this terrain – covered starting in Chapter 2 – does not fix any of these four weaknesses; however, they do add a stronger authentication mechanism on top of an identity infrastructure, that, if the trust architecture, tiering, CA permissions, or hybrid sync hardening are wrong, was already compromisable before the smart card ever entered the picture.

Chapter 2 maps FICAM architecture onto this terrain directly: the PIV/CAC issuance chain, the mechanics of PKINIT smart card logon at the protocol level, and where the certificate mapping logic magic between a physical credential and an AD account breaks down in ways that matter operationally, not theoretically.